

Grille d'évaluation – Dépôt Git et Qualité du code

Évaluation sur 20 points

Cette grille évalue : - La **propreté du dépôt** et sa documentation, - L'**architecture du code** et les bonnes pratiques, - Le **respect du cahier des charges** fonctionnel.

Critères d'évaluation

Critères	Compétences évaluées	Points	Observations / Commentaires
1. Documentation & README	README clair et professionnel avec instructions de lancement (ex: <code>uvicorn app.main:app --reload</code>). Scénario et univers du jeu documentés. Présence de diagrammes de classes (ex: <i>Mermaid</i>).	4 pts	
2. Architecture Cohérente & Bonnes Pratiques	Séparation claire des responsabilités : <code>app/domain/</code> (métier), <code>app/routers/</code> (API REST), <code>app/schemas/</code> (Pydantic). Utilisation systématique des <i>Type Hints</i> et des <i>Dataclasses</i> .	4 pts	

Critères	Compétences évaluées	Points	Observations / Commentaires
3. Code Fonctionnel & Respect du Brief	Code exécutable sans erreur . Implémentation pertinente de l' héritage et du polymorphisme (ex: classes <code>GameElement</code> , <code>Item</code> , <code>Door</code> , et classe abstraite <code>Puzzle</code> déclinée en <code>CodePuzzle</code> et <code>HashPuzzle</code>). Endpoints REST (exploration des salles, soumission des puzzles) fonctionnels .	5 pts	
4. Validation & Sécurité	Schémas Pydantic validant correctement les requêtes (ex: levée d'exception pour un code soumis vide). Logique de vérification cryptographique (SHA-256 / MD5) fonctionnelle dans <code>HashPuzzle</code> .	3 pts	

Critères	Compétences évaluées	Points	Observations / Commentaires
5. Tests Fonctionnels et Unitaires	Présence de tests pertinents . Couverture des méthodes critiques du domaine métier (ex: <code>check_solution</code>) et des routes d'API (avec <code>TestClient</code> de FastAPI). Tous les tests passent.	4 pts	
Total		20 pts	